■ Python APIs for Kids

Requests 101 — Complete Lesson Guide

■ Beginner Friendly	■ ~2 Hours	■ Hands-On

■ What You Will Learn

- What an API is and why it matters
- How the internet communicates using HTTP
- How to install and import the `requests` library
- How to make GET requests and read JSON responses
- How to send POST requests with data
- How to handle errors gracefully
- Three complete mini-projects to practise your skills

■ Table of Contents

Section 1 — What is an API?

API stands for **Application Programming Interface**. That sounds complicated, but the idea is simple: an API is a way for two programs to talk to each other.

> ■ **Real-World Analogy** ■
>
> Think of an API like a waiter at a restaurant. **You** are the customer (your Python program). The **kitchen** is the server (a big computer somewhere on the internet). The **waiter** is the API — it takes your order (request), goes to the kitchen, and brings back your food (response). You never have to go into the kitchen yourself!

APIs are everywhere:

- ■ Weather App — Asks a weather API for today's forecast
- ■ Google Maps — Uses a maps API to show directions
- ■ Spotify — Uses a music API to stream songs
- ■ WhatsApp — Uses messaging APIs to send texts

Key Vocabulary

API	A set of rules for how programs communicate
Client	The program making the request (your Python script)
Server	The computer that listens and responds
Request	A message asking for something
Response	The answer sent back by the server
Endpoint	A specific URL where an API listens

Section 2 — How the Internet Talks (HTTP)

HTTP (HyperText Transfer Protocol) is the language computers use to talk to each other on the web. Every time you open a website, your browser sends an HTTP request and gets an HTTP response.

HTTP Methods

There are four main HTTP methods you need to know:

GET	■ Read data from a server. Like asking 'What's the weather?'
POST	✉■ Send new data to a server. Like submitting a form.
PUT	✏■ Update existing data. Like editing a profile.
DELETE	■ Remove data from a server.

HTTP Status Codes

Every response comes with a **status code** — a number that tells you if things went well or not:

200	**OK**	Everything worked perfectly! ■
201	**Created**	Something new was created (e.g. a new user account) ■
400	**Bad Request**	Your request had an error ■■
401	**Unauthorised**	You need to log in first ■
403	**Forbidden**	You don't have permission ■
404	**Not Found**	That URL doesn't exist ■
500	**Internal Server Error**	Something broke on the server side ■

Section 3 — Setting Up

Installing the `requests` Library

Python doesn't include the `requests` library by default. We need to install it using **pip**, Python's package manager.

Step 1 — Open your terminal or command prompt, then type:

```
pip install requests
```

Step 2 — At the top of every Python file that uses requests, add:

```
import requests
```

> ■ **What is pip?**
>
> **pip** is Python's package installer — like an app store for Python libraries. Type `pip install <package-name>` to install any library.

Checking Your Setup

Run this mini-script to confirm everything is working:

```
import requests

print('requests version:', requests.__version__)
print('Setup is complete! Ready to talk to APIs.')
```

If you see a version number printed (like `2.31.0`), you're all set! ■

Section 4 — Your First GET Request

A **GET request** is the most common type of API call. It's used to *fetch* (retrieve) information from a server — like asking a question.

Basic Syntax

```
import requests

# Make a GET request
response = requests.get('https://api.example.com/data')

# Check if it worked
print(response.status_code) # Should print 200

# See the raw text
print(response.text)
```

Try It — A Real Working Example

The **JSONPlaceholder** API is a free, fake API perfect for learning. Let's fetch a post:

```
import requests

url = 'https://jsonplaceholder.typicode.com/posts/1'

response = requests.get(url)

print('Status Code:', response.status_code)
print('Response:')
print(response.text)
```

■ Expected Output

```
Status Code: 200
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident ...",
  "body": "quia et suscipit ..."
}
```

Exploring the Response Object

`response.status_code`	The HTTP status number (200, 404, etc.)
`response.text`	The response body as a plain string
`response.json()`	Parse the response body as JSON → Python dict
`response.headers`	A dictionary of HTTP headers

`response.url`	The final URL that was called
`response.ok`	True if status code is 200–299

Section 5 — Understanding JSON

JSON (JavaScript Object Notation) is the most popular format for sending data over the internet. It looks very similar to Python dictionaries!

JSON vs Python — Side by Side

JSON

```
{
"name": "Alice",
"age": 12,
"hobbies": ["coding", "art"],
"active": true
}
```

Python Dictionary

```
{
'name': 'Alice',
'age': 12,
'hobbies': ['coding', 'art'],
'active': True
}
```

Topic	In JSON	In Python
Strings	JSON uses double quotes: "hello"	Python allows single OR double: 'hello'
Booleans	JSON: true / false (lowercase)	Python: True / False (capitalised)
Null	JSON: null	Python: None

Parsing JSON with requests

```
import requests

response = requests.get('https://jsonplaceholder.typicode.com/users/1')

# Convert JSON response to a Python dictionary
user = response.json()

# Access individual fields like a dictionary
print('Name:', user['name'])
print('Email:', user['email'])
print('City:', user['address']['city']) # Nested access
```

Section 6 — Query Parameters

Query parameters let you customise your request — like adding filters to a search. They appear after a ? in the URL.

URL Anatomy

https://	api.example.com	/search	?	q=cats&limit;=5
scheme	*host*	*path*		*query parameters*

Instead of building the URL string by hand, pass a `params` dictionary to `requests.get()`:

```
import requests

url = 'https://jsonplaceholder.typicode.com/posts'

# Pass parameters as a dictionary
params = {
'userId': 1, # Only posts by user #1
'_limit': 3, # Only return 3 results
}

response = requests.get(url, params=params)
posts = response.json()

print(f'Got {len(posts)} posts')
for post in posts:
print('-', post['title'][:50])
```

■ Tip

Using the `params` dictionary is safer than building URLs manually because requests will automatically URL-encode special characters for you.

Section 7 — Headers & Authentication

Headers are like sticky notes attached to your request. They carry extra information — what type of data you're sending, who you are, etc.

Common Request Headers

`Content-Type`	Tells the server what format you're sending (e.g. application/json)
`Accept`	Tells the server what format you want back
`Authorization`	Your API key or token (keeps your requests secure)
`User-Agent`	Identifies your application to the server

Sending Headers in requests

```
import requests

headers = {
'Accept': 'application/json',
'User-Agent': 'MyPythonApp/1.0',
}

response = requests.get('https://httpbin.org/headers', headers=headers)
print(response.json())
```

API Key Authentication

Many real APIs require an **API key** — a secret password that proves who you are. Never share your API key with others!

```
import requests

API_KEY = 'your_api_key_here' # Replace with your actual key

# Method 1: API key in headers
headers = {'Authorization': f'Bearer {API_KEY}'}
response = requests.get('https://api.example.com/data', headers=headers)

# Method 2: API key as a query parameter (depends on the API)
params = {'api_key': API_KEY}
response = requests.get('https://api.example.com/data', params=params)
```

■ Security Warning ■

Never write your API key directly in code you share or upload to GitHub! Use **environment variables** or a **.env file** to store secrets safely.

Section 8 — POST Requests

While **GET** fetches data, **POST** sends new data to a server — like filling in a form online. You use `requests.post()` and pass your data in the request body.

Sending JSON Data (most common)

```
import requests

url = 'https://jsonplaceholder.typicode.com/posts'

# Data to send
new_post = {
'title': 'My First API Post',
'body': 'I learned how to use requests in Python!',
'userId': 1
}

# Send as JSON
response = requests.post(url, json=new_post)

print('Status:', response.status_code) # Expect 201 Created
created = response.json()
print('New post ID:', created['id'])
```

Sending Form Data

Some older APIs still accept HTML form data. Use `data=` instead of `json=`:

```
# Sending form-encoded data
form_data = {
'username': 'alice123',
'password': 'secret',
}
response = requests.post('https://example.com/login', data=form_data)
```

GET vs POST — At a Glance

	GET	POST
Purpose	Fetch/read data	Send/create data
	URL (query parameters)	Request body
Bookmarkable	Yes	No
	Yes	Not always (may create duplicates)

Use for	Searching, reading, filtering	Creating accounts, submitting forms

Section 9 — Error Handling

Things don't always go as planned — the server might be down, the URL might be wrong, or you might run out of internet. Good programs handle errors gracefully instead of crashing.

Checking the Status Code

```
import requests

response = requests.get('https://jsonplaceholder.typicode.com/posts/999')

if response.status_code == 200:
data = response.json()
print('Success!', data)
elif response.status_code == 404:
print('Not found — check the URL or ID')
elif response.status_code == 401:
print('Unauthorised — check your API key')
else:
print(f'Something went wrong: {response.status_code}')
```

Using raise_for_status() — The Shortcut

Instead of checking every possible code, call `raise_for_status()` — it automatically raises an error for any 4xx or 5xx response:

```
import requests

try:
response = requests.get('https://jsonplaceholder.typicode.com/posts/1')
response.raise_for_status() # Raises an error if status is 4xx or 5xx

data = response.json()
print('Title:', data['title'])

except requests.exceptions.HTTPError as e:
print(f'HTTP error occurred: {e}')

except requests.exceptions.ConnectionError:
print('Could not connect — check your internet!')

except requests.exceptions.Timeout:
print('The request took too long — try again')

except requests.exceptions.RequestException as e:
print(f'Something went wrong: {e}')
```

Setting a Timeout

Always set a timeout so your program doesn't wait forever if a server is slow:

```
# Wait max 5 seconds for a response
response = requests.get('https://api.example.com/data', timeout=5)

# Or set connect timeout and read timeout separately
response = requests.get('https://api.example.com/data', timeout=(3, 10))
# connect read
```

Section 10 — Mini Projects ■

Project 1 — Weather Reporter

Use the **Open-Meteo** API (completely free, no key needed!) to fetch and display the current temperature for any city.

```
import requests

def get_weather(latitude, longitude, city_name):
url = 'https://api.open-meteo.com/v1/forecast'
params = {
'latitude': latitude,
'longitude': longitude,
'current': 'temperature_2m,wind_speed_10m',
'timezone': 'auto'
}
try:
response = requests.get(url, params=params, timeout=10)
response.raise_for_status()
data = response.json()
current = data['current']
temp = current['temperature_2m']
wind = current['wind_speed_10m']
print(f'Weather in {city_name}:')
print(f' Temperature: {temp} deg C')
print(f' Wind Speed: {wind} km/h')
except requests.exceptions.RequestException as e:
print(f'Error: {e}')

# Try different cities!
get_weather(11.0168, 76.9558, 'Coimbatore')
get_weather(13.0827, 80.2707, 'Chennai')
get_weather(51.5074, -0.1278, 'London')
```

Project 2 — Random Joke Fetcher

Use the **Official Joke API** to fetch and display a random programming joke:

```
import requests

def get_joke():
url = 'https://official-joke-api.appspot.com/jokes/programming/random'
```

```
try:
response = requests.get(url, timeout=10)
response.raise_for_status()
jokes = response.json() # Returns a list
joke = jokes[0]
print('Here is a programming joke for you:')
print()
print('Setup: ', joke['setup'])
print('Punchline:', joke['punchline'])
except Exception as e:
print(f'Could not get joke: {e}')

get_joke()
```

Project 3 — Country Info Lookup

Use the **REST Countries API** to look up facts about any country:

```
import requests

def country_info(country_name):
url = f'https://restcountries.com/v3.1/name/{country_name}'
try:
response = requests.get(url, timeout=10)
response.raise_for_status()
countries = response.json()
c = countries[0] # Take the first match
name = c['name']['common']
capital = c.get('capital', ['N/A'])[0]
population = c.get('population', 0)
region = c.get('region', 'N/A')
languages = list(c.get('languages', {}).values())
print(f'Country: {name}')
print(f'Capital: {capital}')
print(f'Population: {population:,}')
print(f'Region: {region}')
print(f'Languages: {", ".join(languages)}')
except requests.exceptions.HTTPError:
print(f'Country "{country_name}" not found!')
except Exception as e:
print(f'Error: {e}')
```

```
country_info('India')
country_info('Brazil')
```

Section 11 — Quick Reference Card

Install	`pip install requests`	*Terminal command — run once*
Import	`import requests`	*Top of every Python file*
GET	`requests.get(url, params={}, headers={}, timeout=5)`	*Fetch data*
POST JSON	`requests.post(url, json={}, headers={}, timeout=5)`	*Send new data*
Status code	`response.status_code`	*200 = OK, 404 = Not Found*
Body text	`response.text`	*Raw string response*
Parse JSON	`response.json()`	*Returns Python dict/list*
Check OK	`response.ok`	*True if 200-299*
Auto-error	`response.raise_for_status()`	*Raises on 4xx/5xx*
Handle errors	`try/except requests.exceptions.RequestException`	*Catch all requests errors*

Useful Free APIs to Practise With

JSONPlaceholder	`https://jsonplaceholder.typicode.com`	Fake posts, users, todos — great for testing
Open-Meteo	`https://open-meteo.com`	Free weather forecasts, no key needed
REST Countries	`https://restcountries.com`	Country facts, flags, currencies
Official Joke API	`https://official-joke-api.appspot.com`	Programming and general jokes
PokeAPI	`https://pokeapi.co`	Pokemon data — very beginner friendly
NASA APOD	`https://api.nasa.gov`	Astronomy picture of the day (free key)
Open Library	`https://openlibrary.org/developers`	Search millions of books

Section 12 — Practice Exercises

■ Easy (Beginner)

1A	Fetch the list of all users from JSONPlaceholder (`https://jsonplaceholder.typicode.com/users`) and print each user's name and email address.
1B	Use PokeAPI (`https://pokeapi.co/api/v2/pokemon/pikachu`) to fetch Pikachu's data. Print its height, weight, and list its first 5 moves.
1C	Fetch the NASA Astronomy Picture of the Day (`https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY`) and print the title and explanation.

■■ Medium (Intermediate)

2A	Write a function `search_books(title)` that searches Open Library for books matching a title and prints the top 5 results with author names.
2B	Create a Pokémon battle simulator: fetch two Pokémon from PokeAPI, compare their base experience, and declare a 'winner'.
2C	Build a currency converter using a free exchange-rate API (like `https://open.er-api.com/v6/latest/USD`) that converts an amount from one currency to another.

■■■ Hard (Challenge)

3A	Create a **weather dashboard**: ask the user for a city name, use a geocoding API to find the latitude/longitude, then fetch and neatly display the 7-day forecast.
3B	Build an API chaining project: fetch a list of posts from JSONPlaceholder, find the user who wrote the most posts, then fetch and display all comments on that user's posts.
3C	Create a **quiz game** using the Open Trivia DB API (`https://opentdb.com/api.php?amount=5&type;=multiple`) that asks the user questions and keeps score.

■ Congratulations, Future API Developer!

You have completed the **Python APIs for Kids — Requests 101** lesson. You now know how to make GET and POST requests, understand JSON, handle errors, and use real APIs. Keep experimenting — the internet is full of free APIs waiting for you to explore!

- ■ Read the official `requests` docs at **docs.python-requests.org**
- ■ Sign up for a free NASA API key and explore space data
- ■ Try building a small web app with Flask that calls an external API
- ■ Learn about **async requests** with the `aiohttp` library for faster programs
- ■ Study API authentication: OAuth 2.0, JWT tokens, and API keys in environment variables

Happy coding! ■